

Informe de laboratorio de Firewall con IPTables

1. Objetivo

El objetivo de este laboratorio es configurar un firewall usando IP Tables en Linux, aplicando reglas que denieguen conexiones entrantes de ciertas IPs desde el exterior, permitan únicamente respuestas a ICMP (ping) iniciadas desde la red interna, permitan únicamente tráfico HTTP y HTTPS hacia el exterior y bloqueen todo lo demás.

Se ha utilizado un entorno con **dos máquinas virtuales** en VirtualBox: -
Firewall: Ubuntu con IPTables (IP interna: 10.10.10.1, IP externa: 10.0.2.15)
- **Cliente interno:** Ubuntu con conexión a la red interna (IP: 10.10.10.2)

2. Entorno y preparativos

2.1 Configuración de interfaces de red:

- FW:
 - enp0s8: Host-only (INSIDE) 10.10.10.1
 - enp0s3: NAT (OUTSIDE) 10.0.2.15
- Cliente:
 - enp0s8: Host-only (INSIDE) 10.10.10.2

(sé que con un solo adaptador de red ya estaba bien pero cuando se ha dicho ya lo estaba haciendo con dos, así que lo he dejado así)

2.2 Verificación de conectividad básica con ping entre máquinas.

```
(kali㉿kali)-[~]
└─$ ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
^C
— 10.10.10.1 ping statistics —
7 packets transmitted, 0 received, 100% packet loss, time 6149ms

[parrot@parrot]-[~]
└─$ ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=0.242 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=0.401 ms
64 bytes from 10.10.10.2: icmp_seq=3 ttl=64 time=0.277 ms
64 bytes from 10.10.10.2: icmp_seq=4 ttl=64 time=0.271 ms
^C
--- 10.10.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3050ms
rtt min/avg/max/mdev = 0.242/0.297/0.401/0.061 ms
```

2.3 Limpieza de reglas existentes en la FW:

```
sudo iptables -F  
sudo iptables -X  
sudo iptables -Z
```

- Esto asegura que se parte de una configuración limpia.
- -F borra reglas, -X borra cadenas creadas por el usuario, -Z resetea contadores de paquetes/bytes.

3. Configuración del firewall paso a paso

3.1 Activar reenvío de paquetes

```
sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

- sh -c ejecuta el comando que está entre comillas dentro de un shell, permitiendo usar redirecciones (>) con sudo.
- Esto permite que la FW pueda reenviar paquetes entre sus interfaces.

3.2 Configurar NAT (Masquerade)

```
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
```

- -t nat indica que se trabaja sobre la tabla NAT (Network Address Translation).
- -A POSTROUTING añade la regla a la cadena POSTROUTING (después de que se haya determinado la ruta, antes de salir por la interfaz).
- -o enp0s3 indica que aplica a la interfaz externa.
- -j MASQUERADE hace que el tráfico saliente cambie su IP origen a la IP de la FW.
- Esto permite que los clientes internos accedan a Internet usando la IP de la FW.

3.3 Permitir tráfico FORWARD necesario

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

- FORWARD es la cadena que gestiona paquetes que atraviesan la FW entre interfaces.
- -m conntrack usa el módulo de seguimiento de conexiones.
- --ctstate RELATED,ESTABLISHED permite que paquetes que son parte de una conexión existente o relacionados con ella pasen.
- -j ACCEPT acepta el paquete.

3.4 Permitir ICMP (ping) saliente y respuestas

```
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -p icmp --icmp-type 8 -m conntrack --ctstate NEW -j ACCEPT
```

```
sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -p icmp --icmp-type 0 -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

- -p icmp indica que la regla se aplica al protocolo ICMP (ping).
- --icmp-type 8 son echo-request, --icmp-type 0 echo-reply.
- Permite que los clientes internos puedan hacer ping a Internet y recibir respuestas.

3.5 Permitir HTTP/HTTPS

```
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -p tcp -m multiport --dports 80,443 -m conntrack --ctstate NEW -j ACCEPT
```

- -p tcp indica el protocolo TCP.
- -m multiport --dports 80,443 permite múltiples puertos de destino, en este caso HTTP (80) y HTTPS (443).
- Permite navegación web desde la red interna.

3.6 Permitir DNS (UDP 53)

```
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -p udp --dport 53 -m conntrack --ctstate NEW -j ACCEPT
```

- Permite que los clientes resuelvan nombres de dominio usando UDP en el puerto 53.

3.7 Bloquear spoofing desde fuera

```
sudo iptables -A FORWARD -i enp0s3 -s 10.10.10.0/24 -m conntrack --ctstate NEW -j DROP
```

- Spoofing es cuando alguien intenta falsificar su dirección IP de origen.
- Esta regla bloquea paquetes entrantes desde fuera que dicen venir de la red interna.

3.8 Política por defecto

```
sudo iptables -P INPUT DROP  
sudo iptables -P FORWARD DROP  
sudo iptables -P OUTPUT ACCEPT
```

- -P establece la política por defecto de cada cadena.
- Aquí todo lo que no coincida con reglas específicas se bloquea en INPUT y FORWARD, pero OUTPUT se permite.

3.9 Guardar reglas

```
sudo apt install -y iptables-persistent  
sudo sh -c "iptables-save > /etc/iptables/rules.v4"
```

- Guarda las reglas para que persistan tras reinicio.

Conceptos adicionales:

- nc -vz host port: Netcat, prueba conexión TCP al host y puerto indicados.
- sudo iptables -L -v -n --line-numbers: Lista todas las reglas con contadores de paquetes/bytes, sin resolver nombres, y con numeración de líneas.

4. Pruebas realizadas para confirmar que funciona:

1. Ping desde cliente a Internet:

ping 8.8.8.8

Resultado: paquetes recibidos.

2. Curl HTTP desde cliente:

curl -I http://google.com

Resultado: muestra lo que debe mostrar (captura adjunta más abajo)

3. Intento de conexión a puerto no permitido (SSH 22):

nc -vz google.com 22

- Resultado: no se establece, mostrando que reglas de bloqueo funcionan.

```
(kali@kali)-[~]
└─$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=16.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=16.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=254 time=16.9 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=254 time=16.6 ms
^C
— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3012ms
rtt min/avg/max/mdev = 16.564/16.783/16.892/0.128 ms
```

Ping a cliente.

```
(kali@kali)-[~]
└─$ curl -I http://google.com
HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-mQnzSks3tEdbo3mArlg2Dg' 'strict-dynamic' 'report-sample' 'unsafe-ev
al' 'unsafe-inline' https: http:report-uri https://csp.withgoogle.com/csp/gws/other-hp
Date: Wed, 29 Oct 2025 10:45:46 GMT
Expires: Fri, 28 Nov 2025 10:45:46 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
```

Curl a Google.

```
(kali㉿kali)-[~]  
$ nc -vz google.com 22  
DNS fwd/rev mismatch: google.com ≠ mad41s10-in-f14.1e100.net  
google.com [142.250.184.14] 22 (ssh) : Connection refused
```

Netcat a Google puerto 22.

6. Conclusión

Se ha configurado un firewall funcional usando IP Tables que cumple con lo pedido. Se ha permitido tráfico web y ping saliente desde la red interna, se han bloqueado intentos no autorizados, y se ha aplicado NAT para que los clientes internos puedan comunicarse con Internet.